

C++ 类继承完整自学手册

目录

- 1. 继承基础概念
 - 2. 公有继承详解
 - 3. 构造函数与析构函数
 - 4. 虚函数与多态
 - 5. 抽象基类
 - 6. 动态内存分配与继承
 - 7. 实战练习
 - 8. 常见问题解答
-

1. 继承基础概念

1.1 什么是继承？

继承是面向对象编程的核心特性之一，它允许我们基于已有的类（基类）创建新的类（派生类）。

核心思想：

-  代码重用：避免重复编写相同功能的代码
-  扩展功能：在现有功能基础上添加新特性
-  层次结构：建立类之间的等级关系

1.2 基本语法

```
cpp
```

```
// 基类
class Base {
private:
    int privateData;
protected:
    int protectedData;
public:
    int publicData;
    void baseMethod();
};

// 派生类
class Derived : public Base {
private:
    int newData;
public:
    void newMethod();
};
```

1.3 访问控制级别

访问级别	基类内部	派生类内部	外部访问	说明
private	✓	✗	✗	只能在基类内部访问
protected	✓	✓	✗	基类和派生类可访问
public	✓	✓	✓	所有地方都可访问

2. 公有继承详解

2.1 is-a 关系

公有继承建立的是 "是一个" 的关系。

cpp

```
class Animal {
public:
    void eat() { cout << "动物在吃东西" << endl; }
    void sleep() { cout << "动物在睡觉" << endl; }
};

class Dog : public Animal {
public:
    void bark() { cout << "狗在叫" << endl; }
};

// 使用示例
Dog myDog;
myDog.eat(); // 继承自Animal
myDog.sleep(); // 继承自Animal
myDog.bark(); // Dog特有的方法
```

2.2 派生类的特性

派生类继承了什么：

-  基类的数据成员
-  基类的成员函数
-  基类的接口

派生类不继承什么：

-  构造函数
-  析构函数
-  赋值运算符

2.3 实际案例：银行账户系统

cpp

// 基础账户类

```
class BankAccount {  
protected:  
    string accountNumber;  
    string ownerName;  
    double balance;  
  
public:  
    BankAccount(const string& number, const string& name, double bal)  
        : accountNumber(number), ownerName(name), balance(bal) {}  
  
    virtual void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
            cout << "存款成功, 余额: " << balance << endl;  
        }  
    }  
  
    virtual void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
            cout << "取款成功, 余额: " << balance << endl;  
        } else {  
            cout << "余额不足或金额无效" << endl;  
        }  
    }  
  
    virtual void displayInfo() const {  
        cout << "账户: " << accountNumber  
            << " 户主: " << ownerName  
            << " 余额: " << balance << endl;  
    }  
  
    virtual ~BankAccount() {}  
};
```

// 储蓄账户 (派生类)

```
class SavingsAccount : public BankAccount {  
private:  
    double interestRate;  
  
public:  
    SavingsAccount(const string& number, const string& name,  
        double bal, double rate)  
        : BankAccount(number, name, bal), interestRate(rate) {}  
};
```

```
void addInterest() {
    double interest = balance * interestRate;
    balance += interest;
    cout << "利息添加: " << interest << " 新余额: " << balance << endl;
}

void displayInfo() const override {
    cout << "储蓄账户 - ";
    BankAccount::displayInfo();
    cout << "利率: " << interestRate * 100 << "%" << endl;
}
};
```

3. 构造函数与析构函数

3.1 构造函数调用顺序

```
cpp

class Base {
public:
    Base() { cout << "Base构造函数" << endl; }
    ~Base() { cout << "Base析构函数" << endl; }
};

class Derived : public Base {
public:
    Derived() { cout << "Derived构造函数" << endl; }
    ~Derived() { cout << "Derived析构函数" << endl; }
};

// 创建对象时的输出:
// Base构造函数
// Derived构造函数
//
// 销毁对象时的输出:
// Derived析构函数
// Base析构函数
```

3.2 成员初始化列表

```
cpp
```

```

class Student {
protected:
    string name;
    int age;

public:
    Student(const string& n, int a) : name(n), age(a) {
        cout << "Student构造函数" << endl;
    }
};

class GraduateStudent : public Student {
private:
    string researchArea;

public:
    // 正确的方式：使用成员初始化列表调用基类构造函数
    GraduateStudent(const string& n, int a, const string& research)
        : Student(n, a), researchArea(research) {
        cout << "GraduateStudent构造函数" << endl;
    }
};

```

重要规则：

1. 🚀 先调用基类构造函数，再调用派生类构造函数
2. ⬅️ 先调用派生类析构函数，再调用基类析构函数
3. 📄 必须在派生类构造函数的初始化列表中调用基类构造函数

4. 虚函数与多态

4.1 为什么需要虚函数？

cpp

```
class Shape {
public:
    // 非虚函数
    void draw() {
        cout << "绘制基础形状" << endl;
    }
};

class Circle : public Shape {
public:
    void draw() {
        cout << "绘制圆形" << endl;
    }
};

// 问题演示
Shape* shapes[] = {new Circle(), new Shape()};
shapes[0]->draw(); // 输出: "绘制基础形状" (不是我们想要的)
```

4.2 虚函数解决方案

cpp

```
class Shape {
public:
    // 虚函数
    virtual void draw() {
        cout << "绘制基础形状" << endl;
    }

    virtual double area() = 0; // 纯虚函数

    virtual ~Shape() {} // 虚析构函数
};

class Circle : public Shape {
private:
    double radius;

public:
    Circle(double r) : radius(r) {}

    void draw() override {
        cout << "绘制圆形，半径：" << radius << endl;
    }

    double area() override {
        return 3.14159 * radius * radius;
    }
};

class Rectangle : public Shape {
private:
    double width, height;

public:
    Rectangle(double w, double h) : width(w), height(h) {}

    void draw() override {
        cout << "绘制矩形：" << width << "x" << height << endl;
    }

    double area() override {
        return width * height;
    }
};

// 多态演示
void demonstratePolymorphism() {
```



```

Shape* shapes[] = {
    new Circle(5.0),
    new Rectangle(4.0, 6.0)
};

for (int i = 0; i < 2; i++) {
    shapes[i]->draw();    // 调用正确的draw()版本
    cout << "面积: " << shapes[i]->area() << endl;
    delete shapes[i];    // 调用正确的析构函数
}
}

```

4.3 虚函数表机制

```

cpp

// 编译器为每个包含虚函数的类创建虚函数表
class Base {
    virtual void func1() { cout << "Base::func1" << endl; }
    virtual void func2() { cout << "Base::func2" << endl; }
    // 虚函数表: [&Base::func1, &Base::func2]
};

class Derived : public Base {
    void func1() override { cout << "Derived::func1" << endl; }
    virtual void func3() { cout << "Derived::func3" << endl; }
    // 虚函数表: [&Derived::func1, &Base::func2, &Derived::func3]
};

```

5. 抽象基类

5.1 什么是抽象基类？

抽象基类（ABC）是包含至少一个纯虚函数的类，不能直接实例化。

```

cpp

```

// 抽象基类

```
class Vehicle {  
public:  
    virtual void start() = 0; // 纯虚函数  
    virtual void stop() = 0; // 纯虚函数  
    virtual void displayInfo() { // 可以有实现  
        cout << "这是一个交通工具" << endl;  
    }  
    virtual ~Vehicle() {}  
};
```

// 具体派生类

```
class Car : public Vehicle {  
private:  
    string brand;  
    int doors;  
  
public:  
    Car(const string& b, int d) : brand(b), doors(d) {}  
  
    void start() override {  
        cout << brand << "汽车启动，检查" << doors << "个车门" << endl;  
    }  
  
    void stop() override {  
        cout << brand << "汽车停止" << endl;  
    }  
};
```

```
class Motorcycle : public Vehicle {  
private:  
    string brand;  
  
public:  
    Motorcycle(const string& b) : brand(b) {}  
  
    void start() override {  
        cout << brand << "摩托车启动" << endl;  
    }  
  
    void stop() override {  
        cout << brand << "摩托车停止" << endl;  
    }  
};
```

// 使用示例

```
void useVehicles() {  
    // Vehicle v; // 错误! 不能实例化抽象类  
  
    Vehicle* vehicles[] = {  
        new Car("丰田", 4),  
        new Motorcycle("本田")  
    };  
  
    for (auto vehicle : vehicles) {  
        vehicle->start();  
        vehicle->displayInfo();  
        vehicle->stop();  
        delete vehicle;  
    }  
}
```

5.2 抽象基类的设计原则

1. **接口定义**：ABC定义了派生类必须实现的接口
2. **代码重用**：可以在ABC中提供通用实现
3. **多态支持**：通过ABC指针管理不同类型的对象

6. 动态内存分配与继承

6.1 基类使用new的情况

```
cpp
```

```

class DynamicBase {
private:
    char* data;
    int size;

public:
    DynamicBase(const char* str = "default") {
        size = strlen(str) + 1;
        data = new char[size];
        strcpy(data, str);
    }

    // 拷贝构造函数
    DynamicBase(const DynamicBase& other) {
        size = other.size;
        data = new char[size];
        strcpy(data, other.data);
    }

    // 赋值运算符
    DynamicBase& operator=(const DynamicBase& other) {
        if (this != &other) {
            delete[] data; // 释放原有内存
            size = other.size;
            data = new char[size];
            strcpy(data, other.data);
        }
        return *this;
    }

    // 虚析构函数
    virtual ~DynamicBase() {
        delete[] data;
        cout << "DynamicBase析构函数" << endl;
    }

    virtual void display() const {
        cout << "数据: " << data << endl;
    }
};

```

6.2 派生类也使用new的情况

cpp

```

class DynamicDerived : public DynamicBase {
private:
    char* extraData;

public:
    DynamicDerived(const char* str = "default",
                  const char* extra = "extra")
        : DynamicBase(str) {
        extraData = new char[strlen(extra) + 1];
        strcpy(extraData, extra);
    }

    // 拷贝构造函数
    DynamicDerived(const DynamicDerived& other)
        : DynamicBase(other) { // 调用基类拷贝构造函数
        extraData = new char[strlen(other.extraData) + 1];
        strcpy(extraData, other.extraData);
    }

    // 赋值运算符
    DynamicDerived& operator=(const DynamicDerived& other) {
        if (this != &other) {
            DynamicBase::operator=(other); // 调用基类赋值运算符
            delete[] extraData;
            extraData = new char[strlen(other.extraData) + 1];
            strcpy(extraData, other.extraData);
        }
        return *this;
    }

    ~DynamicDerived() {
        delete[] extraData;
        cout << "DynamicDerived析构函数" << endl;
    }

    void display() const override {
        DynamicBase::display();
        cout << "额外数据: " << extraData << endl;
    }
};

```

关键点：

-  派生类构造函数调用基类构造函数
-  派生类赋值运算符调用基类赋值运算符

- 🗑️ 析构函数自动按相反顺序调用
-

7. 实战练习

练习1：员工管理系统

创建一个员工管理系统，包含以下类：

cpp

// 基类：员工

```
class Employee {
protected:
    string name;
    int id;
    double baseSalary;

public:
    Employee(const string& n, int i, double salary)
        : name(n), id(i), baseSalary(salary) {}

    virtual double calculateSalary() const = 0;
    virtual void displayInfo() const {
        cout << "姓名: " << name << " ID: " << id
            << " 基本工资: " << baseSalary << endl;
    }

    virtual ~Employee() {}
};
```

// 派生类：全职员工

```
class FullTimeEmployee : public Employee {
private:
    double bonus;

public:
    FullTimeEmployee(const string& n, int i, double salary, double b)
        : Employee(n, i, salary), bonus(b) {}

    double calculateSalary() const override {
        return baseSalary + bonus;
    }

    void displayInfo() const override {
        cout << "全职员工 - ";
        Employee::displayInfo();
        cout << "奖金: " << bonus << " 总工资: " << calculateSalary() << endl;
    }
};
```

// 派生类：兼职员工

```
class PartTimeEmployee : public Employee {
private:
    int hoursWorked;
    double hourlyRate;
```

```
public:
    PartTimeEmployee(const string& n, int i, int hours, double rate)
        : Employee(n, i, 0), hoursWorked(hours), hourlyRate(rate) {}

    double calculateSalary() const override {
        return hoursWorked * hourlyRate;
    }

    void displayInfo() const override {
        cout << "兼职员工 - ";
        Employee::displayInfo();
        cout << "工作时间: " << hoursWorked
            << " 时薪: " << hourlyRate
            << " 总工资: " << calculateSalary() << endl;
    }
};
```

练习2：图形绘制系统

cpp


```
#include <iostream>
#include <vector>
#include <memory>
using namespace std;
```

```
// 抽象基类
```

```
class Drawable {
public:
    virtual void draw() const = 0;
    virtual double area() const = 0;
    virtual ~Drawable() = default;
};
```

```
// 圆形类
```

```
class Circle : public Drawable {
private:
    double radius;
    string color;

public:
    Circle(double r, const string& c) : radius(r), color(c) {}

    void draw() const override {
        cout << "绘制" << color << "的圆形，半径：" << radius << endl;
    }

    double area() const override {
        return 3.14159 * radius * radius;
    }
};
```

```
// 矩形类
```

```
class Rectangle : public Drawable {
private:
    double width, height;
    string color;

public:
    Rectangle(double w, double h, const string& c)
        : width(w), height(h), color(c) {}

    void draw() const override {
        cout << "绘制" << color << "的矩形：" << width << "x" << height << endl;
    }

    double area() const override {
```

```

        return width * height;
    }
};

// 使用示例
void graphicsDemo() {
    vector<unique_ptr<Drawable>> shapes;

    shapes.push_back(make_unique<Circle>(5.0, "红色"));
    shapes.push_back(make_unique<Rectangle>(4.0, 6.0, "蓝色"));
    shapes.push_back(make_unique<Circle>(3.0, "绿色"));

    cout << "绘制所有图形: " << endl;
    for (const auto& shape : shapes) {
        shape->draw();
        cout << "面积: " << shape->area() << endl << endl;
    }
}

```

8. 常见问题解答

Q1: 什么时候使用虚函数？

A: 当你希望派生类重新定义基类方法，并且要通过基类指针或引用调用派生类版本时。

Q2: 为什么析构函数要声明为虚函数？

A: 防止通过基类指针删除派生类对象时只调用基类析构函数，导致内存泄漏。

```

cpp

Base* ptr = new Derived();
delete ptr; // 如果析构函数不是虚函数，只会调用Base的析构函数

```

Q3: private、protected、public继承的区别？

A:

- **public继承**：is-a关系，基类的public成员在派生类中仍为public
- **protected继承**：基类的public成员在派生类中变为protected
- **private继承**：基类的public和protected成员在派生类中都变为private

Q4: 多重继承要注意什么？

A:

- 菱形继承问题

- 虚继承解决方案
- 构造函数调用顺序
- 二义性问题

Q5: 如何选择使用继承还是组合?

A:

- **继承**: 当存在明确的is-a关系时
 - **组合**: 当存在has-a关系时
-

学习建议

1. **循序渐进**: 从简单的继承开始, 逐步学习虚函数和多态
2. **动手实践**: 每个概念都要编写代码验证
3. **画类图**: 用UML图表示类之间的关系
4. **阅读代码**: 分析开源项目中的继承设计
5. **设计模式**: 学习常见的设计模式

总结

继承是C++面向对象编程的核心概念, 掌握以下要点:

- ☒ 理解is-a关系和继承的本质
- ☒ 掌握构造函数和析构函数的调用顺序
- ☒ 熟练使用虚函数实现多态
- ☒ 理解抽象基类的设计思想
- ☒ 处理动态内存分配的继承问题
- ☒ 遵循良好的设计原则

通过本手册的学习和练习, 你将能够熟练运用C++继承机制设计出清晰、可维护的面向对象程序。